Project Zomboid MCP Server

A comprehensive Model Context Protocol (MCP) server for Project Zomboid mod development, providing intelligent script validation, generation, and contextual assistance through AI-enhanced tooling.

🚀 Features

Smart Project Zomboid Integration

- **Auto-detection** of Steam, Epic Games, and GOG installations
- **Cross-platform support** (Windows, Linux, macOS, WSL)
- **Build 42 compatibility** with modern mod structure support
- **Fallback system** with local script parsing

Comprehensive Game Data Knowledge

- **Complete vanilla game indexing** with full-text search capabilities
- **Rich metadata extraction** including damage, durability, categories, and tags
- **Relationship mapping** between items, recipes, and dependencies
- **Real-time reference validation** against game database

Intelligent Script Generation

- **Template-based generation** using real game patterns
- **Balance analysis** comparing custom items to vanilla equivalents
- **Reference validation** ensuring all dependencies exist
- **Multiple output formats** (items, recipes, fixing scripts, sounds, vehicles)

Advanced Validation Engine

- **Real-time syntax validation** with detailed error reporting
- **Reference checking** for items, sounds, and sprites
- **Balance analysis** with gameplay impact assessment
- **Best practices suggestions** for mod development

Deployment Ready

- **Cloudflare Workers** support for serverless deployment
- **D1 Database** integration for persistent storage
- **HTTP API** for integration with any MCP client
- **Claude Desktop** ready with example configurations

🔧 Installation

Prerequisites

- Node.js 18.0.0 or higher
- npm or yarn package manager

Local Development

```
# Clone the repository
git clone https://github.com/minimax/pz-mcp-server.git
cd pz-mcp-server

# Install dependencies
npm install

# Build the project
npm run build

# Run in development mode
npm run dev
```

Cloudflare Workers Deployment

```
# Install Wrangler CLI
npm install -g wrangler

# Login to Cloudflare
wrangler login

# Create D1 database
wrangler d1 create pz-mcp-prod

# Deploy to Cloudflare Workers
wrangler deploy
```

📖 Usage

With Claude Desktop

Add to your `claude_desktop_config.json`:

```
{
  "mcpServers": {
    "pz-mcp-server": {
      "command": "node",
      "args": ["/path/to/pz-mcp-server/dist/index.js"]
    }
  }
}
```

With Cursor/VSCode

The server can be integrated with any IDE that supports MCP protocol:

1. Install the MCP extension for your IDE
2. Configure the server endpoint
3. Start using Project Zomboid development tools

🛠️ MCP Tools

`search_vanilla`

Search vanilla Project Zomboid content with intelligent matching.

Parameters:
- `query` (string): Search query for game content
- `type` (string, optional): Filter by content type (item, recipe, sound, vehicle)

- `category` (string, optional): Filter by item category
- `limit` (number, optional): Maximum results (default: 20)

Example:

```
// Search for weapons
await mcp.callTool('search_vanilla', {
  query: 'katana',
  type: 'item',
  category: 'Weapon'
});
```

`generate_script`

Generate balanced Project Zomboid scripts using templates and game data.

Parameters:
- `type` (string): Script type (item, recipe, evolvedrecipe, fixing, sound, vehicle)
- `name` (string): Name of the item/recipe to generate
- `properties` (object): Properties and specifications
- `module` (string, optional): Module name (default: "Base")

Example:

```
// Generate a custom weapon
await mcp.callTool('generate_script', {
  type: 'item',
  name: 'SuperKatana',
  properties: {
    DisplayName: 'Super Katana',
    Type: 'Weapon',
    MaxDamage: 5.0,
    Weight: 2.0,
    Categories: 'LongBlade'
  }
});
```

`validate_script`

Validate Project Zomboid script syntax and references with detailed error reporting.

Parameters:
- `content` (string): Script content to validate
- `type` (string, optional): Expected script type
- `strict` (boolean, optional): Enable strict validation mode

Example:

```
// Validate mod script
await mcp.callTool('validate_script', {
  content: scriptContent,
  type: 'item',
  strict: true
});
```

`check_references`

Validate item, sound, and sprite references against game database.

Parameters:
- `references` (string[]): List of references to validate
- `type` (string, optional): Type of references (item, sound, sprite, all)

Example:

```
// Check if items exist
await mcp.callTool('check_references', {
  references: ['Base.Katana', 'Base.Apple'],
  type: 'item'
});
```

`analyze_mod`

Comprehensive analysis of mod directory including balance, compatibility, and structure validation.

Parameters:
- `modPath` (string): Path to mod directory
- `checkBalance` (boolean, optional): Perform balance analysis
- `checkCompatibility` (boolean, optional): Check compatibility with vanilla
- `generateReport` (boolean, optional): Generate detailed analysis report

Example:

```
// Analyze mod quality
await mcp.callTool('analyze_mod', {
  modPath: '/path/to/my-mod',
  checkBalance: true,
  checkCompatibility: true
});
```

`parse_game_files`

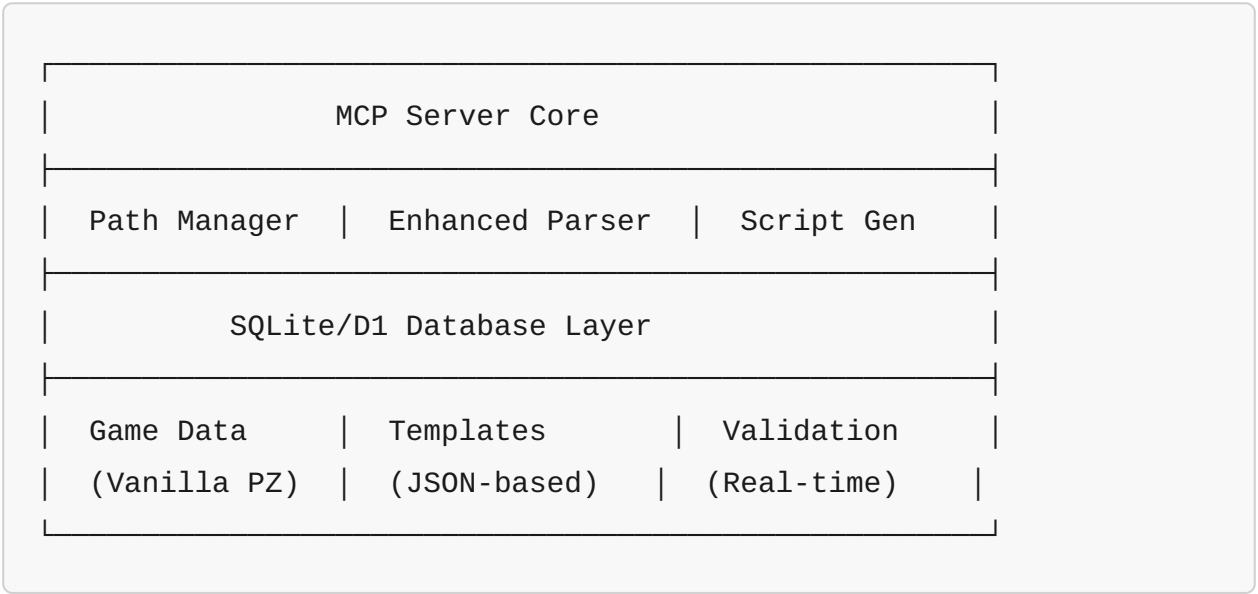
Parse and index Project Zomboid game files to populate the database.

Parameters:
- `gamePath` (string, optional): Path to Project Zomboid installation (auto-detected if not provided)
- `forceReparse` (boolean, optional): Force re-parsing even if data exists

Example:

```
// Parse vanilla game files
await mcp.callTool('parse_game_files', {
  forceReparse: false
});
```

🏗️ Architecture



Core Components

- **DatabaseManager**: SQLite/D1 database with full-text search capabilities
- **ProjectZomboidParser**: Parse vanilla game files and mod directories
- **ScriptGenerator**: Generate balanced scripts using templates and game data
- **ValidationEngine**: Real-time syntax and reference validation
- **ModAnalyzer**: Comprehensive mod analysis and quality metrics

- **PathManager**: Auto-detection of Project Zomboid installations

🌐 Cloudflare Workers Deployment

The server includes full Cloudflare Workers support for serverless deployment:

Features

- **D1 Database** for persistent storage
- **KV Storage** for caching frequently accessed data
- **HTTP API** endpoints for all MCP tools
- **Automatic scaling** with zero cold starts
- **Global edge deployment** for low latency

API Endpoints

- `GET /health` - Health check
- `GET /mcp/info` - Server capabilities
- `POST /tools/{toolName}` - Execute MCP tools
- `POST /admin/load-game-data` - Load vanilla game data

Configuration

Update `wrangler.toml` with your database IDs:

```
[[env.production.d1_databases]]
binding = "DB"
database_name = "pz-mcp-prod"
database_id = "your-database-id"
```

📋 Development Workflow

Setting Up for Mod Development

1. **Initialize Database**:
   `bash npm run dev # Server will auto-detect Project Zomboid installation`
2. **Parse Game Files**:
   `typescript await mcp.callTool('parse_game_files', {});`
3. **Start Development**:
   ```typescript
   // Search for existing items
   const results = await mcp.callTool('search_vanilla', {
   query: 'weapon damage > 3'
   });

// Generate new item
const script = await mcp.callTool('generate_script', {
type: 'item',
name: 'MyWeapon',
properties: { / ... / }
});

// Validate before use
const validation = await mcp.callTool('validate_script', {
content: script
});
```

Supported File Formats

- **mod.info**: Mod metadata and configuration
- **Script Files (.txt)**: Items, recipes, vehicles, sounds, fixing scripts
- **Lua Files (.lua)**: Game logic and event handlers
- **Assets**: Textures, sounds, models, and maps

🔍 Examples

Creating a Custom Weapon

```
// 1. Search for similar weapons
const similarWeapons = await mcp.callTool('search_vanilla', {
  query: 'katana sword blade',
  type: 'item'
});

// 2. Generate balanced weapon
const weaponScript = await mcp.callTool('generate_script', {
  type: 'item',
  name: 'EliteKatana',
  properties: {
    DisplayName: 'Elite Katana',
    Type: 'Weapon',
    Weight: 2.5,
    MaxDamage: 4.5,
    MinDamage: 3.5,
    Categories: 'LongBlade',
    Icon: 'Katana',
    SwingSound: 'KatanaSwing'
  }
});

// 3. Validate the script
const validation = await mcp.callTool('validate_script', {
  content: weaponScript,
  strict: true
});

// 4. Check references exist
await mcp.callTool('check_references', {
  references: ['Katana', 'KatanaSwing'],
  type: 'all'
});
```

Analyzing Mod Quality

```
const analysis = await mcp.callTool('analyze_mod', {
  modPath: '/path/to/my-zombie-mod',
  checkBalance: true,
  checkCompatibility: true,
  generateReport: true
});

console.log(`Mod Quality Score: ${analysis.quality.overall}/100`);
console.log(`Issues Found: ${analysis.issues.length}`);
console.log(`Recommendations: ${analysis.recommendations.join(',
')}`);
```

🤝 Contributing

1. Fork the repository
2. Create a feature branch
3. Make your changes
4. Add tests for new functionality
5. Submit a pull request

📄 License

MIT License - see the LICENSE file for details.

🆘 Support

- **GitHub Issues**: Bug reports and feature requests
- **Documentation**: Comprehensive guides and API references

- **Community**: Discord server for mod developers

🔮 Roadmap

v1.1.0 - Enhanced Features

- **Vehicle script support** with complete parsing and generation
- **Advanced templates** for complex modding scenarios
- **Lua script integration** for game logic assistance
- **Performance optimization** tools for large mods

v1.2.0 - Collaboration Features

- **Multi-user support** for team mod development
- **Version control integration** with Git workflows
- **Automated testing** pipelines for mod validation
- **Documentation generation** from mod analysis

v2.0.0 - Full Platform

- **Web interface** for non-technical users
- **Steam Workshop integration** for direct publishing
- **Marketplace features** for mod discovery
- **Enterprise support** for large mod teams

Built with ❤️ for the Project Zomboid modding community